

UCS802: COMPILER CONSTRUCTION

L	T	P	Cr
3	0	2	4.0

Course Objectives: To Gain the working knowledge of the major phases of compilation and develop the ability to use formal attributed grammars for specifying the syntax and semantics of programming languages. Learn about function and complexities of modern compilers and design a significant portion of a compiler.

Introduction to compiling: Compilers, Analysis of the source program, the phases of Compiler, Compilation and Interpretation, Bootstrapping and Cross compiler.

Lexical Analysis: Need of Lexical analyzer, Tokens and regular expressions, Generation of lexical analyzer from DFA, Introduction to LEX and program writing in LEX.

Syntax Analysis: Need for syntax analysis and its scope, Context free grammar, Top down parsing, bottom up parsing, backtracking and their automatic generation, LL(1) Parser, LR Parser, LR(0) items, SLR(1), LALR(1), Canonical Parsing, Introduction to YACC and Integration with LEX.

Error Analysis: Introduction to error analysis, detection, reporting and recovery from compilation errors, Classification of error-lexical, syntactic and semantic.

Static semantics and Intermediate Code generation: Need for various static semantic analyses in declaration processing, name and scope analysis, S-attribute def. and their evaluation in different parsing, Semantic analysis through S-attribute grammar, L-attribute def. and their evaluation.

Run time Environment: Need for runtime memory management, Address resolution of runtime objects at compile time, Type checking, Language features influencing run time memory management, Parameter passing mechanism, Division of memory into code, stack, heap and static, Activation record, Dynamic memory management, garbage collection.

Code Generation: Code generation for expressions, Issues in efficient code generation, Sethi Ullman algorithm.

Code Optimization: Need for code optimizations, Local and global optimization, Control flow analysis, Data flow analysis, performing global optimizations, Graph coloring in optimization, Live ranges of run time values.

Laboratory work:

Construct a lexical analyzer using Flex. Construct a parser using Bison/ any programming language. Build simple compilers from parsing to intermediate representation to code generation and simple optimization.

Course Learning Outcomes (CLOs) / Course Objectives (COs):

After the completion of the course, the student will be able to:

1. Comprehend the working of major phases of compiler.
2. Apply top-down and bottom-up parsing techniques for the Parser construction.
3. Understand the basic data structures used in compiler construction such as abstract syntax trees, symbol tables and three-address code
4. **Understand target machine's run time environment and techniques used for code generation.**

Text Books:

1. Aho V. A., Ullman D. J., Sethi R. and Lam S. M., Compilers Principles, Techniques and Tools, Pearson Education (2007), 2nd ed.
2. Levine J., Mason T., Brown D., Lex and Yacc, O'Reilly (2012), 2nd ed.